

Frontend Development

HTML architecture, CSS design system and JavaScript logic

Frontend Development Documentation

Project: FitBuddy — AI-Powered Personal Fitness Coach

Overview

The FitBuddy frontend is a **single-page application (SPA)** built with vanilla HTML5, CSS3, and JavaScript. It is served directly by the FastAPI backend at the root endpoint (/) and communicates with the API via `fetch()` calls.

File Structure

```
Frontend/ ├── index.html # Main HTML page (single-page app) ├── static/ ├── style.css # All CSS styles and design system └── app.js # Application logic and API calls
```

1. Page Architecture (`index.html`)

Design Decisions

- **Single-page layout** with tab navigation (no routing required)
- **Semantic HTML5** structure for accessibility
- **CSS custom properties** (variables) for consistent theming
- **ARIA attributes** for accessibility (`role="tablist"`, `aria-selected`)

Sections

Section	ID	Description
Background Orbs	<code>.bg-orbs</code>	Animated gradient orbs for visual depth
Toast Container	<code>#toast-container</code>	Fixed position notification area
Hero Header	<code>.hero-header</code>	Title, badge, stats with animated counters
Navigation	<code>.nav-tabs</code>	3 tab buttons for panel switching
Panel 1	<code>#panel-generate</code>	Profile form + Plan generation + Results
Panel 2	<code>#panel-feedback</code>	Feedback input + Updated plan display
Panel 3	<code>#panel-nutrition</code>	Goal selection + Nutrition tip display
Footer	<code>.footer</code>	Credits and branding

2. CSS Design System (style.css)

Color Palette (CSS Variables)

```
--bg-primary: #0a0e1a /* Deep space dark */ --bg-secondary: #0f1528 /* Slightly lighter dark */
--accent-primary: #6C63FF /* Electric purple */ --accent-secondary: #FF6584 /* Pink-red */
--accent-green: #43E97B /* Vibrant green */ --accent-orange: #FA8231 /* Warm orange */
--accent-cyan: #38EFF0 /* Bright cyan */ --accent-purple: #A855F7 /* Deep purple */
```

Gradients

```
--grad-primary: linear-gradient(135deg, #6C63FF, #A855F7) --grad-green: linear-gradient(135deg,
#43E97B, #38EFF0) --grad-fire: linear-gradient(135deg, #FA8231, #FF6584) --grad-hero:
linear-gradient(135deg, #0a0e1a, #0f1528, #1a0a2e)
```

Typography

- **Primary Font:** Outfit (weights: 300, 400, 500, 600, 700, 800, 900)

- **Secondary Font:** Space Grotesk (weights: 400, 500, 600, 700)
- Imported from Google Fonts

Animations Implemented

Animation	Effect	Used On
orbFloat	Floating up/down	Background orbs
fadeSlideDown	Fade in from top	Hero elements
fadeIn	Fade in from bottom	Section panels
slideInLeft	Slide from left	User ID banner, Toast
slideInRight	Slide from right	Toast notifications
scaleIn	Scale up from center	Results containers
pulse	Pulsing dot	Live indicator badge
spin	Button spinner	Loading states

Component Classes

Class	Component	Description
.card	Card container	Glassmorphism card with blur
.btn-primary	Main CTA	Purple gradient button
.btn-green	Nutrition CTA	Green gradient button
.btn-fire	Feedback CTA	Orange-red gradient button
.goal-card	Goal selector	Clickable goal option
.intensity-btn	Intensity selector	Workout intensity button
.day-card	Day plan card	Expandable day card
.exercise-item	Exercise row	Individual exercise display

<code>.toast</code>	Notification	Slide-in notification
---------------------	--------------	-----------------------

3. JavaScript Application Logic (`app.js`)

State Management

```
const state = {
  currentUserId: null, // Set after plan generation
  currentGoal: '', // Selected goal (weight_loss, etc.)
  currentIntensity: '', // Selected intensity (low, medium, high)
  lastPlan: null, // Last generated plan object
};
```

Core Functions

Function	Description
<code>showToast(msg, type, duration)</code>	Display toast notification
<code>switchTab(tabName)</code>	Navigate between panels
<code>selectGoal(goal)</code>	Handle goal card selection
<code>selectIntensity(intensity)</code>	Handle intensity button selection
<code>applyChip(text)</code>	Insert feedback chip text into textarea
<code>toggleDay(card)</code>	Expand/collapse day cards
<code>updateUserIdFields()</code>	Auto-fill user ID in other panels
<code>generatePlan()</code>	Scenario 1 — fetch and display plan
<code>renderPlan(plan, name)</code>	Render the generated plan HTML
<code>renderDayCard(day, index)</code>	Render individual day card
<code>updatePlan()</code>	Scenario 2 — update plan with feedback
<code>renderFeedbackResult(plan, feedback)</code>	Render updated plan
<code>getNutritionTip()</code>	Scenario 3 — fetch nutrition tip
<code>renderNutritionResult(nutrition)</code>	Render nutrition card

`animateCounters()`

Animate hero stat counters

API Integration

```
const API_BASE = ''; // Same origin = served by FastAPI // Scenario 1: Generate Plan POST
/api/generate-plan Body: { name, age, weight, goal, intensity } // Scenario 2: Update Plan POST
/api/update-plan Body: { user_id, feedback } // Scenario 3: Nutrition Tip POST /api/nutrition-tip
Body: { user_id, goal }
```

Error Handling

- All fetch calls are wrapped in try-catch blocks
- HTTP errors are parsed from JSON response body
- Toast notifications show user-friendly error messages
- Buttons are re-enabled and loading states are cleared in `finally` blocks

4. Responsive Design

Breakpoint	Behavior
Desktop (>900px)	3-column goal grid, 2-column form grid
Tablet (600-900px)	2-column goal grid, 1-column form
Mobile (<600px)	Single column, stacked elements, smaller text

Key responsive classes: `.form-grid`, `.goal-grid`, `.intensity-grid`, `.days-scroll`, `.food-grid`, `.advice-grid`

5. Accessibility Features

- `role="tablist"` and `role="tab"` on navigation
- `aria-selected` on active tab
- `id` attributes on all interactive elements
- Form labels properly linked to inputs with `for` attribute
- Keyboard-accessible buttons throughout

